

From C Source Code to Executable

A practical study of compilation stages using GCC

01 Aim

To examine how GCC transforms and runs a C program, using a program that computes the area of a circle as the working example.

02 Learning Objective

To identify the roles of preprocessing, compilation, assembly, linking, and execution, and to inspect the intermediate files retained by GCC through the `-save-temps` option.

03 Software and Tools

Linux operating system	GCC compiler suite
Command-line terminal	Text editor such as nano or VS Code

04 Program Used

```
#include <stdio.h>

int main()
{
    float radius, area;
    printf("Enter radius of circle: ");
    scanf("%f", &radius);
    area = 3.14159 * radius * radius;
    printf("Area of circle = %.2f\n", area);
    return 0;
}
```

05 Procedure - Part A

1. Prepare the source. Create `circle.c` and type the program shown above.
2. Perform a standard build. Compile the file with GCC. When no output name is supplied, GCC produces the executable `a.out`.
3. Test the executable. Run the program, enter any radius, and observe the area displayed.

05 Procedure - Part B

4. Retain GCC's temporary outputs. Compile again with `gcc -save-temps circle.c`. This keeps the intermediate products that are normally removed.
5. Inspect preprocessing. Open `circle.i`. At this point, directives such as `#include` have been expanded and processed.
6. Inspect generated assembly. View `circle.s`, which contains assembly instructions produced from the preprocessed program.
7. Check the object file. Examine `circle.o`, the machine-code object created by the assembler.
8. Understand linking. The linker combines the object file with the required library code and creates `a.out`.
9. Verify the final build. Execute `./a.out` once more and confirm that the area is correct.

06 Command Reference

```
gcc circle.c
./a.out
gcc -save-temps circle.c
ls
head -n 30 circle.i
head -n 30 circle.s
file circle.o
file a.out
./a.out
```

Key observation: `-save-temps` exposes the products of individual compilation stages without changing the program's intended output.

07 Compilation Stages

Stage	File / Product	What happens
Source	circle.c	The programmer's original C instructions.
Preprocessing	circle.i	Header inclusion and preprocessor directives are resolved.
Compilation	circle.s	Preprocessed C is translated into assembly language.
Assembly	circle.o	Assembly instructions become relocatable machine code.
Linking	a.out	Object code and supporting libraries are joined into an executable.
Execution	Program output	The executable runs and prints the circle's calculated area.

08 Compilation Flow

circle.c	Preprocessor	circle.i	Compiler	circle.s
Assembler	circle.o	Linker	a.out	Execution / Output

Read left to right on the first row, then continue left to right on the second row.

09 Expected Output

```
Enter radius of circle: 5
Area of circle = 78.54
```

The displayed value varies with the input. For a radius of 5 units, the calculated area is approximately 78.54 square units.

10 Screenshots and Evidence

```
(kali㉿kali)-[~]
$ cd ~/Desktop

(kali㉿kali)-[~/Desktop]
$ mkdir Experiment2

(kali㉿kali)-[~/Desktop]
$ cd Experiment2

(kali㉿kali)-[~/Desktop/Experiment2]
$ vi circle.c

(kali㉿kali)-[~/Desktop/Experiment2]
$ cat circle.c
#include <stdio.h>
int main()
{
    float radius, area;
    printf("Enter radius of circle: ");
    scanf("%f", &radius);
    area = 3.14159 * radius * radius;
    printf("Area of circle = %.2f\n",area);
    return 0;
}
```

Figure 1. Experiment2 directory and source program creation.

```
(kali㉿kali)-[~/Desktop/Experiment2]
$ gcc circle.c

(kali㉿kali)-[~/Desktop/Experiment2]
$ ls
a.out circle.c

(kali㉿kali)-[~/Desktop/Experiment2]
$ ./a.out
Enter radius of circle: 5
Area of circle = 78.54
```

Figure 2. Standard GCC compilation followed by program execution.

```
(kali㉿kali)-[~/Desktop/Experiment2]
$ gcc -save-temps circle.c

(kali㉿kali)-[~/Desktop/Experiment2]
$ ls
a-circle.i a-circle.o a-circle.s a.out circle.c
```

Figure 3. Intermediate files created using gcc -save-temps.

10 Screenshots and Evidence - Continued

```
(kali㉿kali)-[~/Desktop/Experiment2]
$ head -n 30 a-circle.i
# 0 "circle.c"
# 0 "<built-in>"
# 0 "<command-line>"
# 1 "/usr/include/stdc-predef.h" 1 3 4
# 0 "<command-line>" 2
# 1 "circle.c"
# 1 "/usr/include/stdio.h" 1 3 4
# 28 "/usr/include/stdio.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/libc-header-start.h" 1 3 4
# 33 "/usr/include/x86_64-linux-gnu/bits/libc-header-start.h" 3 4
# 1 "/usr/include/features.h" 1 3 4
# 415 "/usr/include/features.h" 3 4
# 1 "/usr/include/features-time64.h" 1 3 4
# 20 "/usr/include/features-time64.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/wordsize.h" 1 3 4
# 21 "/usr/include/features-time64.h" 2 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/timesize.h" 1 3 4
# 19 "/usr/include/x86_64-linux-gnu/bits/timesize.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/wordsize.h" 1 3 4
# 20 "/usr/include/x86_64-linux-gnu/bits/timesize.h" 2 3 4
# 22 "/usr/include/features-time64.h" 2 3 4
# 416 "/usr/include/features.h" 2 3 4
# 523 "/usr/include/features.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/sys/cdefs.h" 1 3 4
# 730 "/usr/include/x86_64-linux-gnu/sys/cdefs.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/wordsize.h" 1 3 4
# 731 "/usr/include/x86_64-linux-gnu/sys/cdefs.h" 2 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/long-double.h" 1 3 4
# 732 "/usr/include/x86_64-linux-gnu/sys/cdefs.h" 2 3 4
# 524 "/usr/include/features.h" 2 3 4
```

Figure 4. A portion of the preprocessed file circle.i.

10 Screenshots and Evidence - Continued

```
(kali@kali)-[~/Desktop/Experiment2]
$ head -n 30 a-circle.s
.file "circle.c"
.text
.section      .rodata
.LC0:
.string "Enter radius of circle: "
.LC1:
.string "%f"
.LC3:
.string "Area of circle = %.2f\n"
.text
.globl main
.type main, @function
main:
.LFB0:
.cfi_startproc
pushq %rbp
.cfi_def_cfa_offset 16
.cfi_offset 6, -16
movq %rsp, %rbp
.cfi_def_cfa_register 6
subq $16, %rsp
leaq .LC0(%rip), %rax
movq %rax, %rdi
movl $0, %eax
call printf@PLT
leaq -8(%rbp), %rax
leaq .LC1(%rip), %rdx
movq %rax, %rsi
movq %rdx, %rdi
movl $0, %eax
```

Figure 5. Assembly output stored in circle.s.

```
(kali@kali)-[~/Desktop/Experiment2]
$ ./a-circle
Enter radius of circle: 5
Area of circle = 78.50
$
```

Figure 6. File identification and final execution output.

11 Result

The circle-area program was compiled and executed successfully. Preprocessing, compilation, assembly, and linking were examined through the intermediate files preserved by GCC's `-save-temps` option.

12 Conclusion

A C source file does not become an executable in a single step. It moves through a sequence of transformations: preprocessing creates expanded C code, compilation produces assembly, assembly generates object code, and linking forms the final executable. Retaining temporary files makes each transformation directly observable.

Outcome: The program output was verified, and every major stage in the GCC build pipeline was identified from its corresponding file.